01c — Conditions: Making Decisions (beginner-friendly)

A **condition** is a yes/no question your code asks to decide what to do. Every "if this, do that" is a condition. This file covers **every kind of condition and comparison**, with **when you'd use each**.

> A condition always boils down to `True` or `False`. `if` runs its block only when the answer is `True`.

---

1. The `if` / `elif` / `else` shapes

Just `if` — do something only when true, otherwise nothing:

```
if score >= 50:
    print("pass")
```

`if` / `else` — one of two paths always runs:

```
if score >= 50:
    print("pass")
else:
    print("fail")
```

`if` / `elif` / `else` — pick ONE from several. Python checks top to bottom and stops at the **first** true one:

```
if age < 13:
    group = "child"
elif age < 20:      # only checked if the first was false
    group = "teen"
else:               # none of the above
    group = "adult"
```

> **Order matters!** Because it stops at the first true branch, put the most specific/strongest condition first. (This is exactly the `is_leap` lesson — check `% 400` before `% 100`.)

Scenario: grading, categorizing by range, pricing tiers, any "which bucket does this fall into" question.

2. Comparison operators (the yes/no questions)

```
a == b     # equal?            (TWO equals — one '=' is assignment,
a != b     # NOT equal?
a <  b     # less than?
a >  b     # greater than?
a <= b     # less than OR equal?
a >= b     # greater than OR equal?
```

> **The #1 beginner bug:** = vs ==. `x = 5` *sets* x to 5. `x == 5` *asks* "is x 5?". In an `if`, you always want ==.

Scenario examples:

```
if n % 2 == 0:        # is n even?
if len(s) > 10:       # is the string long?
if x != 0:            # guard before dividing by x
```

3. Combining conditions: and, or, not

```
and   # True only if BOTH sides are true
or    # True if EITHER side is true
not   # flips True<->False
```

```
if age >= 18 and country == "RW":   # must be BOTH
if x < 0 or x > 100:                 # true if EITHER (out of range,
if not found:                        # true when found is False
```

Scenario: validation ("is the input allowed?"), range checks, "either of these two cases".

Plain-English tips:

- `and` = "all conditions must hold" (strict).
- `or` = "any one is enough" (lenient).
- `not` = "the opposite".

4. Chained comparisons (very Pythonic)

Instead of writing two comparisons with `and`, Python lets you chain them:

```
if 0 <= x < 100:          # same as:  x >= 0 and x < 100
if 1 <= day <= 7:         # is day between 1 and 7 inclusive?
```

Scenario: "is this number inside a range?" — grid bounds, valid ages, valid indexes. You already used this in the grid bounds check: `if 0 <= nr < rows`.

---

5. Membership: `in` and `not in`

Asks "is this item inside that collection?"

```
if ch in "aeiou":           # is the character a vowel?
if x in nums:               # is x somewhere in the list?
if key in my_dict:          # does the dict have this key?
if "cat" not in words:      # is "cat" absent?
```

Scenario: checking vowels/allowed characters, "have we seen this?", membership tests.

> **Speed note:** `x in list` is slow on huge lists (re-scans everything). `x in set` or `x in dict` is instant — see file 03. For small inputs, `in` on a list is fine.

---

6. Truthy / falsy — the short way to check "empty" or "zero"

Python treats some values as automatically `False`:

Counts as False	Counts as True
`0`, `0.0`	any other number
`""` (empty string)	any non-empty string
`[]` `{}` `set()` (empty)	any non-empty collection
`None`	most things

So you can write:

```
if nums:            # "if the list is not empty"
    ...
if not s:           # "if the string is empty"
    ...
while n:            # "while n is not 0"  (you used this in count_d
    ...
```

Scenario: guarding against empty input, "keep going while there's something left".

---

7. Conditions as True/False values (predicates)

A function that returns a yes/no answer is a **predicate**. You can return the comparison directly — no `if` needed, because `==`, `<`, etc. already give `True`/`False`:

```
def is_even(n):
    return n % 2 == 0        # already True or False

def is_teen(age):
    return 13 <= age < 20
```

Scenario: `is_valid`, `is_leap`, `is_palindrome`, `contains_x` — any "does this hold?" question.

---

8. Common condition recipes (memorize)

```
n % 2 == 0            # even
n % 2 == 1            # odd  (or:  n % 2 != 0)
n % k == 0            # n divisible by k
0 <= i < len(a)       # i is a valid index
c.isdigit()           # character is a digit
c.isalpha()           # character is a letter
a == a[::-1]          # string/list is a palindrome
x in seen             # we've seen x before (seen is a set)
not nums              # the list is empty
```

---

9. A note on `break`/`continue` with conditions

Conditions team up with loops all the time (see file 01b):

```python
for x in nums:
    if x < 0:
        continue          # condition decides to SKIP this item
    if x == target:
        return True       # condition decides to STOP early
```

Scenario: "skip the zeros", "stop when you find the answer".

Quick "which condition tool?" reference

I want to...	Use
do something only sometimes	`if`
choose between two paths	`if / else`
pick one of several buckets	`if / elif / else` (strongest first)
require several things at once	`and`
accept any one of several	`or`
flip a true/false	`not`
check a number is in a range	`low <= x <= high`
check an item is inside a collection	`in` / `not in`
check a collection is empty	`if not thing:`
return a yes/no answer	`return <comparison>`